Sanskriti Saurabh
Date: 20/3/2026
Superset ID: 6784944

LangGraph Quiz

📌 Section A: MCQs (1 mark each)

1. What is LangGraph primarily used for?
a) Data visualization
b) Building graph-based AI workflows
c) Database management
d) Web scraping

---

2. In LangGraph, nodes represent:
a) Database tables
b) Functions or steps in a workflow
c) UI components
d) API keys

---

3. What is the main advantage of graph-based workflows?
a) Linear execution only
b) No need for state
c) Flexible branching and looping
d) Faster internet speed

---

4. What does “state” represent in LangGraph?
a) Server location
b) Memory shared across nodes
c) Programming language
d) UI design

---

5. Which component decides the next step in a LangGraph workflow?
a) Tokenizer
b) Edge

c) Prompt
d) Dataset

---

6. A multi-step agent is best described as:
a) A single API call
b) A system executing multiple connected steps
c) A database query
d) A static webpage

---

7. What is the purpose of conditional edges in LangGraph?
a) To delete nodes
b) To control flow based on conditions
c) To store data
d) To train models

---

8. Which of the following is required to define a graph in LangGraph?
a) Nodes and edges
b) Only APIs
c) Only datasets
d) Only UI components

---

9. StateGraph in LangGraph is used for:
a) Styling UI
b) Managing workflow state and transitions
c) Creating databases
d) Sending emails

---

10. What is the output of a node in LangGraph?
a) Only text
b) Updated state
c) Images only
d) HTML code

📌 **Section B: Short Answer (2–3 marks each)**

11. Explain the concept of graph-based workflows in LangGraph.
Graph-based workflows represent processes as nodes (steps) and edges (connections).
They allow branching, looping, and dynamic execution, unlike linear pipelines.

---

12. What is state management and why is it important in multi-step agents?
State management means maintaining shared data across all steps in a workflow.
It is important because:
Ensures continuity
Enables multi-step reasoning
Allows decision-making based on past steps

---

13. Differentiate between nodes and edges in LangGraph.

---

14. What is a StateGraph and how does it help in building agents?
1. Define state
2. Manage transitions between nodes
3. Build multi-step AI agents

---

15. Give one real-world example where a multi-step agent can be useful.
Customer support chatbot:
Understand query
Route to billing/technical
Generate response

📌 Section C: Coding-Based Questions (4–5 marks each)

16. Consider the following code:

```
def step1(state):
    return {"msg": "Hello " + state["name"]}
```

👉 What is happening in this function? Explain.
Takes input state
Accesses name

Creates greeting message
Returns **updated state with "msg"**

---

17. Write a simple LangGraph workflow with two steps:

- Step 1: Take user input
- Step 2: Generate a response

(You can write pseudocode)
state={“input”:” “,”response”:””}
def step1(state):
    state[“input”]=user_input()
    return state
Graph:
step1->END

---

18. How would you add a conditional step in a LangGraph workflow? Explain with logic.
if state[“score”]>50:
    go to “pass_node”
else:
    go to “fail_node”

In LangGraph:
1. Use conditional edges
2. Route flow based on state values

📌 Section D: Case Study (5–8 marks)

19. Design a multi-step agent using LangGraph for a “Student Assistant Bot” that:

- Takes a question
- Checks if it is about assignments or exams
- Routes to the correct handler
- Returns a response

👉 Explain:

- Nodes
- Edges

- State

  Nodes

- **Input Node**

  Takes student question

- **Classifier Node**

  Detects type (assignment/exam)

- **Assignment Handler**

  Handles assignment queries

- **Exam Handler**

  Handles exam queries

- **Response Node**

  Returns answer

Input → Classifier

Classifier → Assignment Node (if assignment)

Classifier → Exam Node (if exam)

Assignment/Exam → Response → END

📌 Section E: Advanced Conceptual (5 marks)

20. Compare traditional linear AI pipelines with graph-based agent workflows. Why is LangGraph better for complex systems?
1. Supports complex decision-making
2. Enables multi-step reasoning
3. Allows dynamic routing
4. Maintains state across steps
5. Ideal for agentic AI systems